SCOUT: Supply-Aware Cold-Start Proactive Query Suggestion for Travel Search

Hao Li
hao.li@airbnb.com
Airbnb, Inc.
USA

Shashank Reddy
shashank.reddy@airbnb.com
Airbnb, Inc.
USA

Kedar Bellare
kedar.bellare@airbnb.com
Airbnb, Inc.
USA

Ashish Jain
ashish.jain@airbnb.com
Airbnb, Inc.
USA

Stephanie Moyerman
stephanie.moyerman@airbnb.com
Airbnb, Inc.
USA

Abstract

Generative query suggestion, powered by Large Language Models (LLMs), has become increasingly popular in search and conversational systems to reduce user friction and guide intent formulation. Existing approaches align suggestions with user preferences (e.g., clicks or conversions). This works for open-ended applications like chatbots and personal assistants, where the result space is unconstrained or historical user free-text queries are abundant.

However, applying these methods to travel search presents two limitations. First, travel search is fundamentally constrained by physical inventory; a query (e.g., "romantic beachfront villa") may yield abundant results in Bali but few in Tokyo, so aligning with user preferences is not by itself grounded in what can be offered. Second, travel platforms traditionally rely on faceted search interfaces with no free-text queries. This creates a cold-start problem: without historical query logs there is no demand-side data for alignment, and without a seed query at request time, suggestions must be generated proactively from structured context alone.

To address these challenges, we propose SCOUT, a bootstrapping framework for supply-aware proactive query suggestion. SCOUT overcomes the data gap by substituting missing demand-side user feedback with supply-side system feedback. It treats the search engine as a reinforcement learning environment, deriving a dense reward from the production reranker's query-listing match scores, and optimizes the policy with Group Relative Policy Optimization (GRPO). SCOUT improves inventory match rate (IMR@18) by 12.3% while preserving diversity, matching a compute-intensive best-of-8 policy at zero marginal inference cost and making supply-aware suggestion deployable on a real-time travel search path.

CCS Concepts

• **Information systems** → **Query suggestion**; • **Computing methodologies** → **Reinforcement learning**.

Keywords

Query suggestion, Reinforcement learning, Cold start, Travel search, Large language models

ACM Reference Format:
Hao Li, Shashank Reddy, Kedar Bellare, Ashish Jain, and Stephanie Moyerman. 2026. SCOUT: Supply-Aware Cold-Start Proactive Query Suggestion for Travel Search.

1 Introduction

Search systems are undergoing a fundamental paradigm shift, evolving from rigid keyword matching and faceted filtering toward fluid, natural language interfaces. In this context, Generative Query Suggestion, powered by Large Language Models (LLMs), has emerged as a powerful tool to reduce user friction and proactively guide search intent formulation. State-of-the-art approaches for aligning these generative models heavily rely on Supervised Fine-Tuning (SFT) and Reinforcement Learning from Human Feedback (RLHF) [2, 8, 18]. However, these standard methodologies implicitly assume the availability of massive demand-side datasets, specifically historical free-text queries and user click-through logs.

Applying these standard alignment techniques to travel search introduces two challenges. First is a critical cold-start data gap. Historically, travel platforms have relied on highly structured UI elements, requiring users to input destinations, travel dates, and guest counts via calendars and dropdowns. Consequently, there is an absence of historical free-text query logs to serve as training data. Second, travel search operates under strict physical inventory constraints. Unlike open-domain search where any linguistically coherent response is valid, a travel query suggestion sets an expectation of physical inventory. Generating an appealing but unfulfillable query (e.g., suggesting a "romantic beachfront villa" in a landlocked city) leads to empty search result pages, degrading user experience. Models without supply-awareness actively hallucinate demand.

Naive approaches to supply-aware generation fail to meet production requirements. Simple template-based generation reads as rigid and robotic, lacking the ability to synthesize fluidly with specific travel contexts. Inventory-conditioned prompting is also a non-starter, as a compact, latency-friendly summary of available inventory for arbitrary travel search does not exist at generation time. A serving-time best-of-N strategy—generating multiple candidate suggestions and querying the live search engine to serve the ones with highest inventory—solves the inventory problem but creates a system bottleneck. Issuing N heavy search requests per interaction introduces latency and computational costs that are prohibitive for a real-time travel search engine.

To bridge the gap between generative intent and physical inventory—without historical user data or serve-time latency and cost penalties—our core insight is to substitute missing demand-side user feedback with supply-side system feedback. We operationalize this insight through the following main contributions:

Table 1: Travel search is uniquely inventory-constrained and lacks user-authored free-text queries. Missing historical logs rule out demand-side alignment, and missing request-time inputs rule out seed-based query reformulation.

	inventory-	user-authored free-text query	
	constrained	as input	in logs
web search / chatbot	no	yes	yes
product search	yes	yes	yes
travel search	yes	no	no

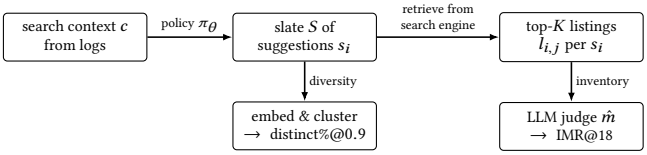


Figure 1: Label-free evaluation pipeline. Historical logs provide the context to generate suggestion slates. We evaluate diversity via embedding-based clustering, and inventory by retrieving the top 18 listings and scoring their semantic match using a calibrated LLM judge.

- **Problem Formulation:** We identify the unique challenge of bootstrapping free-text query suggestions for travel search. Due to the absence of historical logs and seed queries, coupled with physical inventory constraints, we formulate this task as proactive query generation directly from search contexts. (§2)
- **Framework:** We propose SCOUT, a bootstrapping framework that leverages the live search engine as an RL training environment. It enables UX-driven evaluation without demand-side data and online RL training driven by supply-side feedback. (§3)
- **Results:** We demonstrate that aligning models via supply-side RL achieves superior supply-awareness while maintaining linguistic diversity compared to existing baselines. Crucially, SCOUT amortizes the prohibitive costs of serve-time validation directly into the model's weights, enabling production deployment. (§4)

2 Problem Setting & Related Work

The Travel Search Setting. Travel platforms have historically relied on structured, faceted search interfaces (date pickers, destination autocomplete) rather than an open text box. Introducing free-text search therefore starts from a setting with two properties that, together, are unique among the domains where generative query suggestion has been studied (Table 1). First, there is no user-authored free-text query: not as input at request time, and not in historical logs. Second, travel search is bounded by physical, location-specific inventory. These two properties are what rule out the two established families of prior work.

Gap 1: Alignment Requires Demand-Side Data. Large Language Models have shifted the query suggestion space toward generative query suggestion, where supervised fine-tuning and RLHF align suggestions with user intent using historical free-text queries and engagement feedback [2, 4, 8, 18]. These methods assume the demand-side data that our setting does not have.

Gap 2: Retrieval Feedback Requires a Seed Query. A second family uses the retrieval system as the reward source, optimizing query reformulation, rewriting, or expansion against retrieval outcomes [1, 6, 9, 10, 17, 19]. While we share the intuition of using the search engine as an RL environment, these existing methods are fundamentally *reactive transformations*. They rely on a user-authored seed query as a reference anchor, optimizing how well the rewrite preserves and expands that existing intent. In contrast, our task is *proactive generation*. Because the free-text query is strictly the output rather than the input, we have no reference intent to anchor the reward. Instead of preserving intent, our model must proactively construct plausible intents directly from structured context and ground them in retrievable inventory.

Formulating Proactive Generation. We define the task as follows: a search context $c = (\ell, d, g)$ is the structured input travel search already accepts—destination ℓ, date range d, guest count g. Given c, a policy π_θ generates a slate of free-text suggestions $S = \{s_1, \ldots, s_{|S|}\}$, surfaced *proactively*, before the user issues any free-text input, since free-text search is a capability the interface has never offered and users must be shown it to adopt it.

3 Methodology

To tackle the absence of demand-side data and the physical constraints of travel search, we introduce a supply-aware framework for generative query suggestion. First, we establish a scalable, label-free evaluation suite to objectively measure inventory and diversity (§3.1). Second, we utilize the live search engine for supply-side feedback, training the policy via online RL (§3.2).

3.1 Evaluation without Demand-Side Data

Two axes: inventory and diversity. Click-through and conversion are unavailable during bootstrapping. To overcome this, we walk backward from the ideal guest experience to define proxy criteria for success. A suggestion sets an expectation about the page behind it, which gives the first axis, *inventory*: clicking must open results that match the suggestion, not an empty or low inventory page. The second axis is *diversity*, both a UX necessity to prevent repetitive slates and a crucial guardrail against diversity collapse, a documented failure mode of preference optimization [13, 14].

Two metrics, one per axis. Because no free-text queries exist, we use historical parameterized searches as contexts and score the slate each policy generates from them (Figure 1). For inventory we define the *Inventory Match Rate*, IMR@18. For every suggestion s_i in a slate S we retrieve the top $K = 18$ listings $l_{i,1}, \ldots, l_{i,K}$, the first page of results, and average a judge's per-listing attribute match ratio $\hat{m}(s_i, l_{i,j}) \in [0, 1]$ over the slate:

$$\text{IMR@18}(S) = \frac{1}{|S|\,K} \sum_{i=1}^{|S|} \sum_{j=1}^{K} \hat{m}(s_i, l_{i,j}).$$

For diversity we track distinct%@0.9, corpus-level distinctness per seed: we embed each suggestion with MiniLM [16], greedily leader-cluster at cosine similarity 0.9, and report $100\times|\text{clusters}|/|\text{suggestions}|$.

A calibrated judge makes inventory measurable at scale. Because human annotation is unscalable, we rely on LLM judges [20]. We first hand-curate a representative "golden set" of query-to-listing matches labeled by human experts. We then construct an

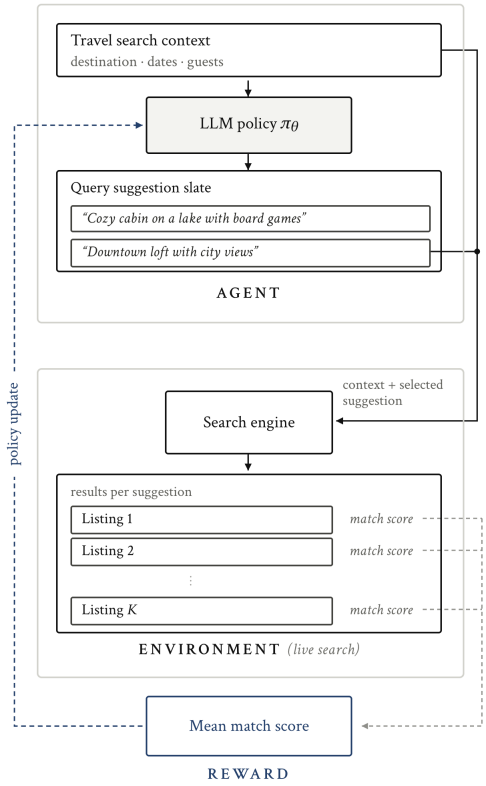


Figure 2: Overview of SCOUT. The LLM generates query suggestions, which are executed against the live search engine; the returned inventory yields the reward, and GRPO internalizes supply-awareness into the policy weights.

LLM judge and iteratively refine its prompt against this golden set. Once the judge's agreement with human labels reaches a pre-set threshold, we deploy it to compute IMR@18 automatically. This calibrated pipeline provides a rigorous, scalable, and label-free compass for evaluating our models prior to online deployment.

3.2 SCOUT: Supply-Aware Optimization via Online RL

The Search Engine as a Reward Environment. The best-of-N policy of §1 can guarantee supply-backed suggestions, but it pays for that guarantee at every impression: N search calls per slate, none of which leave the underlying model any more supply-aware. The cost recurs indefinitely because the knowledge stays outside the model. SCOUT instead pays it once, at training time, by treating the live search stack as a reinforcement learning environment and amortizing inventory validation into the policy weights.

To achieve this without historical demand-side data, we formulate query generation as a reinforcement learning problem where the live search engine acts as the environment (Figure 2). The LLM agent observes a state c (the parameterized travel context, such as destination and dates) and takes an action by generating a slate of query suggestions S. These suggestions are executed against the environment (the search engine), which retrieves physical inventory and returns a reward based on the semantic quality of those listings, driving the policy update.

Supply-Aware Reward Design. While our canonical evaluation metric, IMR@18, provides a high-fidelity assessment of inventory, computing it during online training is computationally intractable. It issues one judge call per query-listing pair, so with $G = 8$ sampled slates per prompt, $|S| = 2$ suggestions per slate, and $K = 18$ retrieved listings per suggestion, a single gradient step would block on $G \cdot |S| \cdot K = 288$ independent LLM calls. Across thousands of steps this is prohibitively slow and expensive.

We therefore define R_{18}, a proxy read directly off the search engine's existing reranker. Let $m(s_i, l_{i,j}) \in [0, 1]$ be the reranker's query-listing semantic match score for the j-th listing retrieved by suggestion s_i, a component that deliberately isolates topical relevance from engagement and booking propensity. The slate-level reward is the mean match over all retrieved listings:

$$R_{18}(S) = \frac{1}{|S|\,K} \sum_{i=1}^{|S|} \sum_{j=1}^{K} m(s_i, l_{i,j}).$$

Crucially, this signal is free. The reranker already scores every listing it returns in the serving path, so all 288 per-listing scores arrive as a byproduct of the $G \cdot |S| = 16$ retrieval calls the rollouts make anyway, so the reward adds no model inference.

Over 9,000 suggestions across 300 contexts, R_{18} and IMR@18 exhibit a stable positive rank correlation (Spearman corr = 0.45, 95% CI $[0.43, 0.48]$, bootstrapped over contexts), justifying its use as a dense reward signal.

Online Policy Update via GRPO. To actively explore the inventory distribution, SCOUT optimizes the policy π_θ using Group Relative Policy Optimization (GRPO) [15]. For each context c, we sample a group of G slates $\{S^{(1)}, \ldots, S^{(G)}\}$. Each slate is executed against the search engine to obtain rewards $\{r^{(1)}, \ldots, r^{(G)}\}$, where $r^{(i)} = R_{18}(S^{(i)})$. We compute a group-relative advantage $A^{(i)}$ by standardizing the rewards within the group:

$$A^{(i)} = \frac{r^{(i)} - \mu}{\sigma}, \quad \text{where } \mu = \frac{1}{G}\sum_{i=1}^{G} r^{(i)}, \quad \sigma = \sqrt{\frac{1}{G}\sum_{i=1}^{G}(r^{(i)} - \mu)^2} \tag{1}$$

Using these advantages, SCOUT updates the policy by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}_{c,\{S^{(i)}\}_{i=1}^{G}} \left[\frac{1}{G} \sum_{i=1}^{G} \min\left(\rho^{(i)} A^{(i)}, \text{clip}(\rho^{(i)}, 1-\epsilon, 1+\epsilon) A^{(i)} \right) - \beta \mathbb{D}_{KL}(\pi_\theta || \pi_{ref}) \right] \tag{2}$$

where $\rho^{(i)} = \frac{\pi_\theta(S^{(i)}|c)}{\pi_{\theta_{old}}(S^{(i)}|c)}$ is the probability ratio, ϵ controls clipping, β weights the KL penalty, and π_{ref} is the reference (base) policy.

4 Results

4.1 Experimental Setup

Setup. All models are initialized from the Qwen3-8B base model [11] and trained using LoRA [5] (rank = 32, $\alpha = 64$, targeting all linear layers). To evaluate generalization to unseen markets, the training and evaluation datasets are disjointly split by destination. We evaluate all methods under the same setting used in training: $|S| = 2$ suggestions per slate, $K = 18$ listings per suggestion. Every trained arm is held to the same training budget and evaluated across three random seeds; we report the seed mean.

Table 2: Main results, relative to Few-shot for inventory and absolute for diversity (three-seed means). SCOUT attains the best inventory grounding at parity diversity. *DPO's diversity collapse makes its inventory deltas not meaningful.

	Inventory		Diversity
Method	ΔR_{18}	ΔIMR@18	distinct%@0.9
Few-shot	–	–	84.03
RSFT	+5.24%	+1.75%	84.11
DPO	n/a*	n/a*	11.07
RPO	+18.12%	+6.02%	84.04
SCOUT	+37.63%	+12.30%	83.72

Baselines. We compare SCOUT against three paradigms:

- **Few-shot (Prompting):** The base model is explicitly instructed to optimize for both diversity and inventory, provided with positive and negative examples. It isolates what training adds over instruction.
- **RSFT (Rejection-Sampling Fine-Tuning) [3]:** We sample $N = 16$ slates per context from the Few-shot policy, score them using the R_{18} metric, and retain the top-1 slate per context for supervised fine-tuning (SFT). RSFT utilizes the identical reward signal as SCOUT but consumes it via imitation rather than RL.
- **DPO / RPO (Preference Optimization):** We construct preference pairs (y^+, y^-) such that the chosen response strictly improves inventory by a margin without degrading diversity ($\Delta_{inv} \geq \delta_{inv}$ and $\Delta_{div} \geq 0$). We apply a per-context cap to prevent any single context from dominating the pair set. DPO [12] is trained with $\beta = 0.1$. RPO [7] utilizes the same setup but introduces an auxiliary SFT loss (pref_ftx = 1.0).

SCOUT (Ours): We run GRPO as described in §3.2 with group size $G = 8$, KL penalty $\beta = 0.06$, fully on-policy (`num_iterations` = 1), initialized from the base model.

4.2 Main Results

The performance of all methods is summarized in Table 2.

The Limits of Prompting and SFT: The Few-shot baseline achieves strong diversity (84.03 distinct%@0.9) but yields only modest inventory grounding. This asymmetry defines the problem for every trained arm: raise inventory-awareness, hold diversity. Prompting struggles because the model inherently lacks access to the dynamic inventory of a given market. RSFT attempts to close this gap by learning from best-of-N generations, but yields only marginal gains (+1.75% IMR@18) because RSFT only copies winning samples: it never observes a good-versus-bad contrast, and remains constrained by the frozen Few-shot policy's output distribution.

The Diversity Collapse in Preference Learning: DPO explicitly learns from contrastive pairs, but we observe a severe diversity collapse (11.07 distinct%@0.9). The model concentrates its distribution on a narrow set of generic queries due to likelihood displacement [13, 14]. RPO mitigates this by anchoring the model with an auxiliary SFT loss. It restores diversity to 84.04 and yields +18.12% R_{18} / +6.02% IMR@18, showing the contrast is informative once the collapse is mitigated.

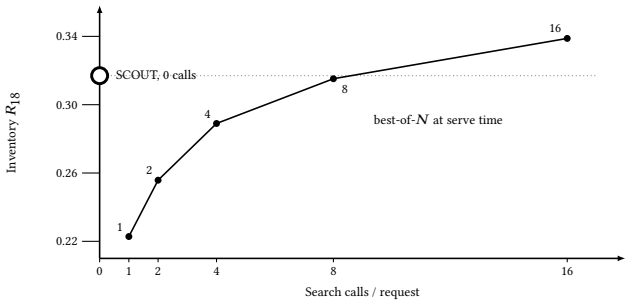


Figure 3: Inventory-awareness vs. serve-time cost. SCOUT achieves the inventory grounding of a best-of-8 policy at zero marginal serve-time cost.

SCOUT Achieves a Balance: SCOUT (GRPO) outperforms all baselines, achieving the highest inventory-awareness (+37.63% R_{18}, +12.30% IMR@18) while maintaining diversity. By optimizing over groups of generations rather than static pairs, SCOUT explores the reward landscape and learns robust supply-aware behaviors.

4.3 Amortizing Serve-Time Search into Model Weights

Figure 3 places these gains against the alternative of validating inventory at serve time. Best-of-N buys inventory-awareness with search calls; each call is a full search request issued on every impression, so the cost recurs for the lifetime of the feature. SCOUT pays it once during training instead: it reaches 0.32 with no search calls of its own, marginally above best-of-8 and 81% of the way from best-of-1 to best-of-16. This is what makes supply-aware query generation deployable on a real-time travel search path.

5 Conclusion and Ongoing Work

We introduced SCOUT, a framework for generating supply-aware query suggestions to bootstrap natural language interfaces. As a short paper on ongoing work, our limitations and future roadmap are best understood through a two-phase deployment strategy:

Phase 1: System Capability Bootstrapping (Current). SCOUT explicitly tackles the cold-start problem by aligning LLMs with supply-side constraints. Consequently, our current limitations include a reliance on offline evaluation metrics and a reward signal that scores fulfillability but not appeal. However, this pure supply-aware bootstrapping is a necessary precondition; it ensures the initial system is reliable and functional enough to launch, which is required to collect meaningful user interaction data.

Phase 2: Demand-Aware Alignment (Ongoing). An ideal query suggestion system must synthesize both supply availability and demand preferences. As the interface rolls out, our ongoing work focuses on large-scale online A/B testing to capture live demand-side signals (e.g., click-through rates, booking conversions). This data will drive our transition from pure supply-awareness to a balanced, multi-objective alignment strategy, ensuring future suggestions are physically fulfillable and resonant with user intent.

References

[1] Sanjay Agrawal, Srujana Merugu, and Vivek Sembium. 2023. Enhancing E-commerce Product Search through Reinforcement Learning-Powered Query Reformulation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM '23)*. ACM, Birmingham, United Kingdom, 4488–4494. doi:10.1145/3583780.3615474

[2] Jinwen Chen, Shiwen Zhang, Shuai Gong, Zheng Zhang, Yachao Zhao, Lingxiang Wang, Haibo Zhou, Wei Lin, and Hainan Zhang. 2026. LocalSUG: City-Preference-Enhanced LLM for Query Suggestion in Local-Life Services. In *Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing: Industry Track*. To appear. arXiv:2603.04946.

[3] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. 2023. RAFT: Reward rAnked FineTuning for Generative Foundation Model Alignment. *Transactions on Machine Learning Research (TMLR)* (2023). arXiv:2304.06767.

[4] Xian Guo, Ben Chen, Siyuan Wang, Ying Yang, Mingyue Cheng, Chenyi Lei, Yuqing Ding, and Han Li. 2026. OneSug: The Unified End-to-End Generative Framework for E-commerce Query Suggestion. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 40. 14774–14782. doi:10.1609/aaai.v40i17.38497

[5] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations (ICLR)*. arXiv:2106.09685.

[6] Pengcheng Jiang, Jiacheng Lin, Lang Cao, Runchu Tian, SeongKu Kang, Zifeng Wang, Jimeng Sun, and Jiawei Han. 2025. DeepRetrieval: Hacking Real Search Engines and Retrievers with Large Language Models via Reinforcement Learning. In *Conference on Language Modeling (COLM)*. arXiv:2503.00223.

[7] Zhihan Liu, Miao Lu, Shenao Zhang, Boyi Liu, Hongyi Guo, Yingxiang Yang, Jose Blanchet, and Zhaoran Wang. 2024. Provably Mitigating Overoptimization in RLHF: Your SFT Loss is Implicitly an Adversarial Regularizer. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 37. 138663–138697. doi:10.52202/079017-4399

[8] Erxue Min, Hsiu-Yuan Huang, Xihong Yang, Min Yang, Xin Jia, Yunfang Wu, Hengyi Cai, Junfeng Wang, Shuaiqiang Wang, and Dawei Yin. 2025. CTR-Guided Generative Query Suggestion in Conversational Search. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Association for Computational Linguistics, Suzhou, China, 2624–2634. doi:10.18653/v1/2025.emnlp-industry.178

[9] Rodrigo Nogueira and Kyunghyun Cho. 2017. Task-Oriented Query Reformulation with Reinforcement Learning. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Copenhagen, Denmark, 574–583. doi:10.18653/v1/D17-1061

[10] Wenjun Peng, Guiyang Li, Yue Jiang, Zilong Wang, Dan Ou, Xiaoyi Zeng, Derong Xu, Tong Xu, and Enhong Chen. 2024. Large Language Model based Long-tail Query Rewriting in Taobao Search. In *Companion Proceedings of the ACM Web Conference 2024 (WWW '24)*. ACM, Singapore, 20–28. doi:10.1145/3589335.3648298

[11] Qwen Team. 2025. Qwen3 Technical Report. arXiv:2505.09388 [cs.CL]

[12] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36. 53728–53741. doi:10.52202/075280-2338

[13] Noam Razin, Sadhika Malladi, Adithya Bhaskar, Danqi Chen, Sanjeev Arora, and Boris Hanin. 2025. Unintentional Unalignment: Likelihood Displacement in Direct Preference Optimization. In *International Conference on Learning Representations (ICLR)*.

[14] Yi Ren and Danica J. Sutherland. 2025. Learning Dynamics of LLM Finetuning. In *International Conference on Learning Representations (ICLR)*.

[15] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300 [cs.CL]

[16] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 5776–5788. arXiv:2002.10957.

[17] Adam Yang, Gustavo Penha, Enrico Palumbo, and Hugues Bouchard. 2025. Aligned Query Expansion: Efficient Query Expansion for Information Retrieval through LLM Alignment. arXiv:2507.11042 [cs.IR]

[18] Junhao Yin, Haolin Wang, Peng Bao, Ju Xu, and Yongliang Wang. 2026. From Clicks to Preference: A Multi-stage Alignment Framework for Generative Query Suggestion in Conversational System. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26)*. ACM, Jeju, Republic of Korea, 2539–2550. doi:10.1145/3770854.3783953

[19] Yipeng Zhang, Bowen Liu, Xiaoshuang Zhang, Aritra Mandal, Canran Xu, and Zhe Wu. 2026. REFLEX: Reinforcement Feedback Learning with Large Language Models for E-commerce Query Expansion. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining (WSDM '26)*. ACM, Boise, ID, USA, 1288–1292. doi:10.1145/3773966.3779381

[20] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, Vol. 36. 46595–46623. doi:10.52202/075280-2020